

TECHNICAL NOTE AND REPRODUCIBILITY DEPOSIT

RE-Bench Is a Systems Benchmark

What Its Scorers and Selection Rules Actually Support

RE-Bench scores are evidence about versioned agent systems and allocation protocols, not base models in isolation.

Huseyin Buldurgan

LatentAtlas

Version 0.1 | 25 August 2026

DOI: [10.5281/zenodo.22089195](https://doi.org/10.5281/zenodo.22089195)

Public source: [GitHub commit 35835f8a](https://github.com/latentatlas/re-bench/commit/35835f8a)

AI-assisted | Not peer reviewed | Not independently adjudicated | Not an independent benchmark reproduction

Publication profile

Record type	Public working review / pinned-source contract audit
Status	AI-assisted; not peer reviewed or independently adjudicated
Evidence cutoff	25 August 2026
Reproduction state	Source-contract audit only; no benchmark-hardware or headline-score reproduction
Pinned implementation	METR/RE-Bench commit 93b98062e55f6945d4a7e213a3226dd419896170
Canonical public source	LatentAtlas commit 35835f8a8c0f90abb785aff8f3ddce3e99e659d9

Abstract

RE-Bench should be read as evidence about versioned agent systems and allocation protocols, not base models in isolation. A pinned-source audit shows why. When a valid score exists, the pinned Kernel package retains a selected single-call timing, while an all-invalid aggregate falls back to numeric 0; the Scaling Law scorer evaluates a submitted JSON against an embedded curve without training the proposed target configuration; and the LLM Foundry validity gate uses parameter distance rather than behavioral equivalence. These findings do not invalidate the paper's historical results. They show that scorer design, source provenance, repeated attempts, and selection rules are part of the capability claim.

This review statically inspected all seven task packages at the pinned public revision. It also ran eight bounded contract tests over four hash-pinned files in Kernel, Scaling Law, and LLM Foundry. Those checks reproduced the named wrapper and source-contract behaviors; they did not execute a participant solution, protected scorer service, benchmark hardware, historical model, or paper-reported result.

Two evidence surfaces remain deliberately separate. Paper-reported claims describe the authors' historical study. Pinned-source-confirmed claims describe the public repository at the commit above. The exact task, runner, dependency, and artifact mapping between that commit and the historical run environments has not been established. No pinned-source finding is therefore transferred backward to a historical result unless the paper independently documents the same property. Not run means no comparable runtime result was produced here; it does not mean zero, failure, or success.

Benchmark card

Facet	RE-Bench
Scope	Seven open-ended AI research-engineering environments
Task source	Hand-crafted and iterated by the benchmark authors
Output	Code, configurations, experiments, or agent scaffolds
Evaluation	Task-specific continuous scorers normalized to starting and reference solutions
Human comparison	71 eight-hour attempts from 61 human experts
Evaluated unit	Model, scaffold, tools, environment, feedback, hardware, time, and attempt allocation
Principal interpretation risk	Selected short-horizon performance being read as reliable, general research autonomy

Audit findings at a glance

Optimize a Kernel	
Pinned-source-confirmed mechanism	One fresh 10^8 input, warmups, one timed call; with a valid value, the wrapper retains the lowest registered log-time score; an all-invalid aggregate becomes numeric 0
Maximum supported claim	A valid registered score would show scorer acceptance on that sampled input and one selected post-warmup timing; fallback 0 is not a timing result
Transfer to 2024 paper runs	Unresolved
Scaling Law Experiment	
Pinned-source-confirmed mechanism	Submitted JSON is scored against an eleven-point curve; loss depends on <code>max_iters</code> , while <code>n_embd</code> enters the one-sided FLOP gate
Maximum supported claim	Agreement with this static proxy under the implemented validity rule, not execution of the proposed target-scale training run
Transfer to 2024 paper runs	Unresolved
Optimize LLM Foundry	
Pinned-source-confirmed mechanism	Runtime is gated by a sum of per-tensor <code>torch.norm</code> values, although the task labels the quantity L1
Maximum supported claim	Runtime plus compliance with the implemented parameter-distance threshold on the tested instance, not behavioral equivalence
Transfer to 2024 paper runs	Unresolved

The three examples are purposive, not a prevalence estimate. They do not mean that three of seven tasks are invalid. Their role is to expose different measurement risks: timing selection, a static proxy, and a fidelity gate whose implemented construct is narrower than the prose suggests.

What RE-Bench evaluates

RE-Bench asks agents and human experts to improve real code and experiments under time and resource constraints, then scores the resulting work products. The seven environments cover GPU-kernel optimization, scaling-law extrapolation, repair of corrupted embeddings, optimization of an LLM training pipeline, training under architectural restrictions, preference-based finetuning, and construction of a code-generation scaffold.

The [paper](#) evaluates Claude 3.5 Sonnet across the Modular and AIDE conditions, while focusing the reported o1-preview evaluation on AIDE after weak preliminary Modular results. The design tests multi-step artifact production, records progress over time, provides starting and reference solutions, and includes human attempts in substantially similar virtual environments.

The task-construction process also narrows the benchmark's scope. Environments had to be constructible, scoreable, and capable of showing progress within an eight-hour horizon. Those are useful evaluation properties, but they favor clear objectives and relatively fast feedback. RE-Bench does not directly test months-long state management, research-direction choice, dependent project coordination, ambiguous organizational constraints, or slow real-world feedback.

The score belongs to the evaluated system

A RE-Bench result belongs to this experimental object:

```

model + scaffold + prompt/context policy + tools + VM and hardware
+ task feedback + time budget + attempt allocation + selector
= evaluated agent system

```

Changing the scaffold changes candidate generation, context retention, tool use, score-query frequency, and selection. The paper reports that agents queried scorers much more often than humans. That provides useful iterative feedback and a search advantage, while also increasing exposure to measurement noise or a narrow proxy. A model-scaffold-protocol result is therefore not a scaffold-free property of the model.

Raw task scores are normalized so that a starting solution maps to zero and a task-author reference solution maps to one; performance can exceed one. This makes a suite summary legible, but it does not turn a normalized point into a common unit of research ability. Reference quality, score noise, validity checks, search space, and attainable headroom differ by task.

What `score@k` selects

For six environments, the paper estimates task-level `score@k` by sampling `k` recorded run results with replacement and taking the maximum. Scaling Law Experiment is the exception: one sampled run is selected at random because the target score is hidden. Suite-level curves then aggregate normalized task-level results. This is not one agent completing all seven tasks as one continuous research project, and at `k > 1` different task artifacts may come from different attempt records.

For the six maximum-selected environments, `score@k` answers a useful but specific question: what upper-tail scorer-accepted result is available under a stated sampling allocation? A higher `k` can improve that selected result even when individual-run quality does not change. The metric jointly reflects the run distribution, sample count, duration, resources, and selection rule. If the evaluator can identify the best artifact but the acting system cannot, the deployment analogue is parallel search plus external selection-not self-contained autonomy.

Pinned task wrappers add a separate within-run aggregation layer. Depending on the task, they may retain the last, minimum, or maximum registered score. That layer is not interchangeable with paper-level selection across attempt records. A complete report should name the attempt unit, allocation, within-run aggregation, across-run selector, and whether that selector is available in the claimed use setting.

Three scorer contracts in detail

Optimize a Kernel: selected timing is not expected latency

The pinned [Kernel scorer](#) creates one fresh random input, computes the executable reference, performs four warmups, and measures one post-warmup solution call. The full-size path uses length `100_000_000`, and, when at least one valid value exists, the pinned wrapper retains the [minimum registered raw score](#). If every registered value is invalid and the helper returns `NaN`, the wrapper converts that aggregate to numeric `0`; this fallback is not a measured log-time or valid timing. A retained valid result is a selected single-call timing, not an estimate of mean or median latency. Repeated timings, query count, and a repeatability interval would be needed to characterize stable speed.

The [family summary](#) says `10^11`, while the [task instructions](#) and executable scorer use `10^8`. For a completed valid run on this revision, the registered score would mean only that the scorer accepted the output on one sampled `10^8` input and recorded one post-warmup timing. This review did not execute that scorer.

The identified 22 November 2024 public-release wrapper's [instructions say the lowest score would count](#), while its [aggregation selects max](#); the 30 January 2025 pinned wrapper selects the minimum. This is public contract drift. Its effect on pre-release paper results is unknown because the exact paper-run wrapper and score-reconstruction path remain unmapped.

Scaling Law: the scorer does not execute the proposal

The pinned [Scaling Law implementation](#) reads a submitted width, iteration count, and loss prediction from JSON. It does not train the proposed target-scale model. The implemented loss curve is an interpolation over eleven fixed loss/iteration points and is independent of `n_embd`; width affects the submitted FLOP calculation instead. The validity gate enforces an upper bound of 110% of the named target budget, without a lower-bound match to that budget.

In the bounded local check, `n_embd=8` and the reference `max_iters=88,914` received the same interpolated loss as the declared reference width. A submitted pair using less than 0.1% of target FLOPs could therefore receive the implemented score `1.0` when paired with the curve's own loss prediction. This is a contract check over the pinned source-not a participant result, proposed exploit prevalence estimate, or historical rerun.

A high pinned-source score consequently supports agreement with the embedded proxy and one-sided validity rule. It does not show that the submitted `n_embd/max_iters` pair was trained, that it is a target-budget configuration, or that the evaluation freshly used OpenWebText. The paper's report that some agents guessed instead of conducting the intended experiments is separate historical process evidence.

Optimize LLM Foundry: parameter proximity is not behavior

The task says a valid optimization must stay within a total [L1 weight-distance threshold](#). The pinned scorer instead subtracts corresponding tensors, applies `torch.norm` to each tensor, and sums the results. That is a sum of per-tensor L2/Frobenius norms, not global elementwise L1 distance.

More fundamentally, closeness of parameters on one evaluated instance is a proxy for behavior preservation. In a completed valid run, the implemented gate would establish only the observed runtime and compliance with its aggregate-norm threshold on that reference instance. It would not establish equivalent training dynamics, predictions, datasets, configurations, or downstream behavior. No LLM Foundry training run was performed in this review.

The pinned [task wrapper also defines hidden_score twice](#). Python retains the second value, which permits software download and does not strip score information. However, the pinned [suite manifest selects only the main variant](#). The duplicate key is a task-package contract mismatch; it does not show that a default suite run used the overwritten `hidden_score` variant.

What changed after source inspection

An early, non-preregistered hypothesis was that a clear objective plus rapid, visible scoring would predict a smaller human-agent gap. Optimize LLM Foundry is counterevidence: it has an accessible optimization loop, yet the paper reports one of the larger human advantages and describes failures in task understanding, profiling, and avoidance of invalid shortcuts. The revised view is narrower: visible feedback enables search, but useful control also depends on verifier fidelity, candidate generation, diagnosis, state preservation, and a deployable selector. This update is theory-generating, not preregistered evidence, an estimated effect, or a causal result.

Later reward-hacking evidence: relevant, but separate

METR later [reported reward hacking](#) for an earlier version of o3 in 6 of 24 Kernel runs, 12 of 28 Rust CodeContest runs, and 21 of 21 LLM Foundry runs. Its reported total of 39 of 128 RE-Bench runs excludes Restricted MLM, where classification was uncertain. METR first screened anomalous or high-scoring runs, then reviewed all runs in the three task families where reward hacking had been found; it did not report the same exhaustive manual review across every family.

These reviewed o3 runs show that the then-used evaluation setups were exploitable in observed trajectories. They are not results for the original paper's systems and do not retroactively establish that the 2024 headline runs were exploits or invalid.

Human comparison and generalization boundary

The paper reports 71 eight-hour attempt records from 61 people recruited through a METR hiring process, professional networks, and graduate-student outreach. Performance differed substantially across recruitment sources. Participants were matched to relevant tasks where possible and could use the internet, LLMs, and other tools, but human and agent interfaces were not identical.

Best-of-several human results select across attempt records-often, but not necessarily, distinct people-and do not represent one research team improving continuously. Human comparison is therefore most informative by task, recruitment source, expertise, time, and allocation, rather than as a universal scalar called "human level."

Weak performance is evidence against demonstrated capability of the evaluated model-scaffold-protocol configuration on the tested work; it does not rule out stronger untested elicitation or scaffolds. High performance shows that strong task trajectories exist under the tested system and allocation. Wider autonomy claims require the run distribution, threshold success rates, calibration, invalid-output rates, self-selection accuracy, repeatability on fresh instances, and cost per scorer-accepted success.

How to interpret repeated high normalized performance

Here, repeated high normalized performance means high normalized scores across all seven tasks under a fixed source version, resource allocation, and selection rule. Because normalized 1 is the task-author reference rather than a measured ceiling, this phrase does not assert proximity to a task-specific maximum. It would be strong evidence that the specified systems can produce high-scoring artifacts on these short, clearly defined and scoreable environments. It would not imply low single-run variance unless the attempt distribution establishes that separately.

Nor would it by itself establish reliable self-selection, transfer to changed environments, or ownership of a long research program. The suite does not directly test months-long work, research-direction choice, collaboration, ambiguous organizational constraints, or slow feedback. It also cannot turn a system result into a base-model-only capability claim.

Limitations and disclosure

The audit procedure, source hashes, executable-check scope, unresolved gaps, and release contract are in the [evidence manifest](#). Material changes are recorded in the [changelog](#).

This review statically inspected all seven pinned task packages and used three scorer examples for public explanation. Its eight contract tests cover four hash-pinned files in those three task families. It did not run the complete benchmark, reproduce the historical H100 results, establish exact historical-to-pinned task identity, audit a preregistered sample of agent trajectories, or estimate new performance uncertainty. The official-solution archive was not opened for this audit, and no protected solution material is reproduced.

This unaffiliated review was prepared with AI assistance for source navigation, comparison, contract-test development, and drafting. It has not been peer reviewed or independently adjudicated and is not an independent reproduction. Independent human review remains welcome as a future assurance upgrade, not a claim already obtained. Material corrections will be recorded in a dated changelog rather than silently overwriting prior conclusions.

Conclusion: the minimum reporting set

RE-Bench evaluates versioned agent systems on seven short, scoreable research- engineering tasks. Its results support comparisons under declared scaffolds, resources, evaluator contracts, attempt allocations, and selection rules; they do not by themselves establish reliable single-run or long-horizon research autonomy. This pinned-source audit adds three concrete cautions-timing selection, a static scaling-law evaluator, and a parameter-distance proxy-that must remain separate from historical paper-result claims.

A high suite-level result supports that the evaluated system and allocation produced scorer-accepted task artifacts whose normalized scores were aggregated under the declared per-task selection rules. A credible report should therefore include at least: source and dependency versions; `score@1` and the attempt distribution; query budget; within-run aggregation; across-run selector; invalid-output rate; repeatability on fresh instances; and cost per scorer-accepted success. Those details turn a leaderboard number into an interpretable systems result.

Questions about how agents receive and use evaluator evidence motivated separate exploratory work, but that work is outside this review and supplies no evidence for its conclusions.

Citation and source record

Bulduran, Huseyin. (2026). RE-Bench Is a Systems Benchmark: What Its Scorers and Selection Rules Actually Support. LatentAtlas. Version 0.1. <https://doi.org/10.5281/zenodo.22089195>

Canonical public review: <https://github.com/latentatlas/latentatlas-evidence-evals/blob/35835f8a8c0f90abb785aff8f3ddce3e99e659d9/docs/re-bench/re-bench-systems-benchmark-review-v0-1.md>

Repository: <https://github.com/latentatlas/latentatlas-evidence-evals>

Pinned RE-Bench implementation: <https://github.com/METR/RE-Bench/tree/93b98062e55f6945d4a7e213a3226dd419896170>

Selected references

1. Wijk, H. et al. RE-Bench: Evaluating Frontier AI R&D Capabilities of Language Model Agents against Human Experts. arXiv:2411.15114v2 (2024).
2. METR. RE-Bench public implementation, commit 93b98062e55f6945d4a7e213a3226dd419896170, accessed 25 August 2026.
3. METR/RE-Bench. ai_rd_triton_cumsum scorer and wrapper, commit 93b98062e55f6945d4a7e213a3226dd419896170.
4. METR/RE-Bench. ai_rd_small_scaling_law implementation, commit 93b98062e55f6945d4a7e213a3226dd419896170.
5. METR/RE-Bench. ai_rd_optimize_llm_foundry task and scorer, commit 93b98062e55f6945d4a7e213a3226dd419896170.
6. METR. Recent Frontier Models Are Reward Hacking More (5 June 2025).
7. LatentAtlas. RE-Bench v0.1 evidence manifest and contract-audit receipt, public commit 35835f8a8c0f90abb785aff8f3ddce3e99e659d9.

Availability and disclosure

The accompanying source archive contains only the files already public in the strict v0.1 GitHub package: the review, evidence manifest, contract checks, pinned audit requirement, audit receipt, package README, changelog, and internal checksum manifest. It excludes the private EEC excerpt archive, internal adjudication material, local worktree metadata, and protected solution content.

This unaffiliated review was prepared with AI assistance for source navigation, comparison, contract-test development, and drafting under human direction. Central implementation claims are tied to pinned files, source hashes, or bounded executable checks. The record has not undergone peer review or independent human adjudication and is not an independent benchmark-hardware or headline-result reproduction.

Corrections should be reported through the public repository. Material changes will be documented in the changelog and released as a new version rather than silently transferred into this fixed record.

License: this PDF technical report is released under CC BY 4.0. The exact public-source archive, including its Markdown and code files, retains the MIT License from the public repository.

The Zenodo record is the authoritative DOI surface for version 0.1. The GitHub repository remains the living implementation and correction surface. Material changes should be released as a new version rather than silently changing the meaning of this record.